Practical – 1

Title:

To perform Selection Sort, Bubble Sort and Insertion Sort on a sufficiently large input and measure their execution time.

Objective:

To study and compare the performance of Selection Sort, Bubble Sort and Insertion Sort algorithms by measuring their execution time for a large set of input data.

Theory:

Sorting is the process of arranging data elements in a specific order such as ascending or descending. Sorting algorithms are widely used in computer science to improve the efficiency of searching and data processing.

Selection Sort

Selection Sort works by repeatedly finding the minimum element from the unsorted part of the array and placing it at the beginning. It performs comparisons even if the array is already sorted.

Time Complexity:

- Best Case: $O(n^2)$
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$

Bubble Sort

Bubble Sort compares adjacent elements and swaps them if they are in the wrong order. This process is repeated until the list is sorted. Larger elements “bubble up” to the end of the array.

Time Complexity:

- Best Case: $O(n)$
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$

Insertion Sort

Insertion Sort works like sorting playing cards in hand. Each element is picked and placed at its correct position in the already sorted part of the array. It is efficient for small or nearly sorted data.

Time Complexity:

- Best Case: O(n)
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$

Algorithm / Steps:

Algorithm for Selection Sort

1. Start from the first element of the array.
2. Assume the first element as the minimum.
3. Compare the minimum element with the remaining elements.
4. If a smaller element is found, update the minimum.
5. Swap the minimum element with the first element.
6. Repeat the process for the remaining elements until the array is sorted.

Algorithm for Bubble Sort

1. Start from the first element of the array.
2. Compare adjacent elements.
3. Swap them if they are in wrong order.
4. After each pass, the largest element moves to the end.
5. Repeat the process until no swaps are required.

Algorithm for Insertion Sort

1. Consider the first element as sorted.
2. Take the next element and compare it with the sorted part.
3. Shift larger elements one position to the right.
4. Insert the element at its correct position.
5. Repeat until the entire array is sorted.

BUBBLE SORT :-

```
#include <stdio.h>

void bubbleSort(int arr[], int n)
{
    int swaps = 0;
    int passes = 0;

    for (int i = 0; i < n - 1; i++)
    {
        int swapped = 0;
        passes++;

        printf("\n[PASS %d] ", i + 1);

        for (int j = 0; j < n - i - 1; j++)
        {
            if (arr[j] > arr[j + 1])
            {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;

                swapped = 1;

                swaps++;
            }
        }

        printf("\nARRAY = ");
        for (int k = 0; k < n; k++) {
            printf("%d ", arr[k]);
        }
```

```
        if (swapped == 0) break;
    }

    printf("\n\n[TOTAL PASSES] = %d", passes);
    printf("\n[TOTAL SWAPS] = %d\n", swaps);
}

int main()
{
    int array[] = {14, 3, 19, 7, 2, 11, 6, 18, 1, 10, 5, 15, 9, 4, 13};
    int size = 15;

    bubbleSort(array, size);
    return 0;
}
```

Bubble Sort Output :-

```
PS D:\ADA-Code\Practical-1> gcc 1.c
PS D:\ADA-Code\Practical-1> ./a.exe

[PASS 1]
ARRAY = 3 14 7 2 11 6 18 1 10 5 15 9 4 13 19
[PASS 2]
ARRAY = 3 7 2 11 6 14 1 10 5 15 9 4 13 18 19
[PASS 3]
ARRAY = 3 2 7 6 11 1 10 5 14 9 4 13 15 18 19
[PASS 4]
ARRAY = 2 3 6 7 1 10 5 11 9 4 13 14 15 18 19
[PASS 5]
ARRAY = 2 3 6 1 7 5 10 9 4 11 13 14 15 18 19
[PASS 6]
ARRAY = 2 3 1 6 5 7 9 4 10 11 13 14 15 18 19
[PASS 7]
ARRAY = 2 1 3 5 6 7 4 9 10 11 13 14 15 18 19
[PASS 8]
ARRAY = 1 2 3 5 6 4 7 9 10 11 13 14 15 18 19
[PASS 9]
ARRAY = 1 2 3 5 4 6 7 9 10 11 13 14 15 18 19
[PASS 10]
ARRAY = 1 2 3 4 5 6 7 9 10 11 13 14 15 18 19
[PASS 11]
ARRAY = 1 2 3 4 5 6 7 9 10 11 13 14 15 18 19

[TOTAL PASSES] = 11
[TOTAL SWAPS] = 55
PS D:\ADA-Code\Practical-1>
```

SELECTION SORT :-

```
#include <stdio.h>

void selectionSort(int arr[], int n)
{
    int swaps = 0;
    int passes = 0;

    for (int i = 0; i < n - 1; i++)
    {
        int minIndex = i;
        passes++;

        printf("\n[PASS %d]", i + 1);

        for (int j = i + 1; j < n; j++)
        {
            if (arr[j] < arr[minIndex])
            {
                minIndex = j;
            }
        }

        if (minIndex != i)
        {
            int temp = arr[i];
            arr[i] = arr[minIndex];
            arr[minIndex] = temp;
            swaps++;
        }

        printf("\nARRAY = ");

        for (int k = 0; k < n; k++)
```

```
        {
            printf("%d ", arr[k]);
        }
        printf("\n");
    }

    printf("\n[TOTAL PASSES] = %d", passes);
    printf("\n[TOTAL SWAPS] = %d\n", swaps);
}

int main()
{
    int array[] = {14, 3, 19, 7, 2, 11, 6, 18, 1, 10, 5, 15, 9, 4, 13};
    int size = 15;

    selectionSort(array, size);
    return 0;
}
```

Selection Sort Output :-

```
PS D:\ADA-Code\Practical-1> gcc 2.c
PS D:\ADA-Code\Practical-1> ./a.exe

[PASS 1]
ARRAY = 1 3 19 7 2 11 6 18 14 10 5 15 9 4 13

[PASS 2]
ARRAY = 1 2 19 7 3 11 6 18 14 10 5 15 9 4 13

[PASS 3]
ARRAY = 1 2 3 7 19 11 6 18 14 10 5 15 9 4 13

[PASS 4]
ARRAY = 1 2 3 4 19 11 6 18 14 10 5 15 9 7 13

[PASS 5]
ARRAY = 1 2 3 4 5 11 6 18 14 10 19 15 9 7 13

[PASS 6]
ARRAY = 1 2 3 4 5 6 11 18 14 10 19 15 9 7 13

[PASS 7]
ARRAY = 1 2 3 4 5 6 7 18 14 10 19 15 9 11 13

[PASS 8]
ARRAY = 1 2 3 4 5 6 7 9 14 10 19 15 18 11 13

[PASS 9]
ARRAY = 1 2 3 4 5 6 7 9 10 14 19 15 18 11 13
```

```
[PASS 10]
ARRAY = 1 2 3 4 5 6 7 9 10 11 19 15 18 14 13

[PASS 11]
ARRAY = 1 2 3 4 5 6 7 9 10 11 13 15 18 14 19

[PASS 12]
ARRAY = 1 2 3 4 5 6 7 9 10 11 13 14 18 15 19

[PASS 13]
ARRAY = 1 2 3 4 5 6 7 9 10 11 13 14 15 18 19

[PASS 14]
ARRAY = 1 2 3 4 5 6 7 9 10 11 13 14 15 18 19

[TOTAL PASSES] = 14
[TOTAL SWAPS] = 13
PS D:\ADA-Code\Practical-1>
```

INSERTION SORT :-

```
#include <stdio.h>

void insertionSort(int arr[], int n)
{
    int passes = 0;
    int shifts = 0;

    for (int i = 1; i < n; i++)
    {
        int key = arr[i];
        int j = i - 1;
        int passShifts = 0;

        passes++;

        printf("\n[PASS %d]", passes);

        while (j >= 0 && arr[j] > key)
        {
            arr[j + 1] = arr[j];
            j--;
            shifts++;
            passShifts++;
        }

        arr[j + 1] = key;

        printf("\nARRAY = ");
        for (int k = 0; k < n; k++)
        {
            printf("%d ", arr[k]);
        }
```

```
        printf("\nSHIFTS IN THIS PASS = %d\n", passShifts);
    }

    printf("\n[TOTAL PASSES] = %d", passes);
    printf("\n[TOTAL SHIFTS] = %d\n", shifts);
}

int main()
{
    int array[] = {14, 3, 19, 7, 2, 11, 6, 18, 1, 10, 5, 15, 9, 4, 13};
    int size = 15;

    insertionSort(array, size);
    return 0;
}
```

Insertion Sort Output :-

```
PS D:\ADA-Code\Practical-1> gcc 3.c
PS D:\ADA-Code\Practical-1> ./a.exe

[PASS 1]
ARRAY = 3 14 19 7 2 11 6 18 1 10 5 15 9 4 13
SHIFTS IN THIS PASS = 1

[PASS 2]
ARRAY = 3 14 19 7 2 11 6 18 1 10 5 15 9 4 13
SHIFTS IN THIS PASS = 0

[PASS 3]
ARRAY = 3 7 14 19 2 11 6 18 1 10 5 15 9 4 13
SHIFTS IN THIS PASS = 2

[PASS 4]
ARRAY = 2 3 7 14 19 11 6 18 1 10 5 15 9 4 13
SHIFTS IN THIS PASS = 4

[PASS 5]
ARRAY = 2 3 7 11 14 19 6 18 1 10 5 15 9 4 13
SHIFTS IN THIS PASS = 2

[PASS 6]
ARRAY = 2 3 6 7 11 14 19 18 1 10 5 15 9 4 13
SHIFTS IN THIS PASS = 4

[PASS 7]
ARRAY = 2 3 6 7 11 14 18 19 1 10 5 15 9 4 13
SHIFTS IN THIS PASS = 1
```

```
[PASS 8]
ARRAY = 1 2 3 6 7 11 14 18 19 10 5 15 9 4 13
SHIFTS IN THIS PASS = 8

[PASS 9]
ARRAY = 1 2 3 6 7 10 11 14 18 19 5 15 9 4 13
SHIFTS IN THIS PASS = 4

[PASS 10]
ARRAY = 1 2 3 5 6 7 10 11 14 18 19 15 9 4 13
SHIFTS IN THIS PASS = 7

[PASS 11]
ARRAY = 1 2 3 5 6 7 10 11 14 15 18 19 9 4 13
SHIFTS IN THIS PASS = 2

[PASS 12]
ARRAY = 1 2 3 5 6 7 9 10 11 14 15 18 19 4 13
SHIFTS IN THIS PASS = 6

[PASS 13]
ARRAY = 1 2 3 4 5 6 7 9 10 11 14 15 18 19 13
SHIFTS IN THIS PASS = 10

[PASS 14]
ARRAY = 1 2 3 4 5 6 7 9 10 11 13 14 15 18 19
SHIFTS IN THIS PASS = 4

[TOTAL PASSES] = 14
[TOTAL SHIFTS] = 55
PS D:\ADA-Code\Practical-1>
```

Practical – 2

Title:

To perform linear and binary search on a sufficiently large input and measure their execution time.

Objective:

To study and compare the performance of linear and binary search algorithms by measuring their execution time for a large set of input data.

Theory:

Linear Search

Linear search is the most straightforward searching algorithm. It starts at the beginning of the list and compares the target value with each element sequentially until a match is found or the end of the list is reached.

- **Pre-condition:** None. Works on both sorted and unsorted arrays.
- **Time Complexity:** O(n), where n$ is the number of elements.
- **Best For:** Small datasets or unsorted data.
-

Binary Search

Binary search is a "divide and conquer" algorithm. It works by repeatedly dividing the search interval in half. If the target value is less than the middle element, it narrows the interval to the lower half; otherwise, it narrows it to the upper half.

- **Pre-condition:** The array **must be sorted**.
- **Time Complexity:** O(log n).
- **Best For:** Large datasets where efficiency is critical.

Algorithm / Steps:

Algorithm for Linear Search:

1. **Start** at the first element (index 0).
2. **Compare** the current element with the target value.
3. **If Match:** Return the current index and exit.
4. **If No Match:** Move to the next element in the array.
5. **Repeat** steps 2–4 until the target is found or the end of the array is reached.
6. **End:** If the loop finishes without a match, return -1 (indicating the element isn't there).

Algorithm for Binary Search:

Steps:

1. **Initialize: Set low to 0 and high to the last index of the array ($n-1$).**
2. **Find Midpoint: Calculate the middle index:**

mid = low + (high – low)/2

3. **Compare:**
   - **Case A: If arr[mid] is equal to the target, return mid.**
   - **Case B: If arr[mid] is less than the target, the target must be in the right half. Set low = mid + 1.**
   - **Case C: If arr[mid] is greater than the target, the target must be in the left half. Set high = mid - 1.**
4. **Repeat: Go back to Step 2 as long as low is less than or equal to high.**
5. **End: If low exceeds high, the target is not in the array. Return -1.**

LINEAR SEARCH :-

```
#include <stdio.h>

int main() {
    int a[15] = {29, 10, 14, 37, 13, 7, 25, 18, 3, 40, 21, 6, 31, 12, 2};
    int found = -1;
    int target;
    printf("Enetr Element you want to search : ");
    scanf("%d", &target);

    for (int i = 0; i < 15; i++) {
        if (a[i] == target) {
            found = i;
            break;
        }
    }

    if (found != -1) {
        printf("Linear Search: Element found at index %d\n", found);
    } else {
        printf("Element not found.\n");
    }

}
```

Linear Search Output :-

```
PS D:\ADA-Code\Practical-2> gcc linearsearch.c
PS D:\ADA-Code\Practical-2> ./a.exe
Enter Element you want to search : 31
Linear Search: Element found at index 12
PS D:\ADA-Code\Practical-2>
```

BINARY SEARCH :-

```
#include <stdio.h>

void binarySearchInAscendingArray (int arr[], int n) {

    for (int i = 0; i < n; i++)
    {
        for (int j = i + 1; j < n; j++)
        {
            int temp;
            if (arr[i] > arr[j]) {
                temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
    }

    printf("\nAscending Sorted Array : ");
```

```
for (int i = 0; i < n; i++)
{
    printf("%d ", arr[i]);
}

printf("\n");

int key;
printf("Enter value of Key : ");
scanf("%d", &key);

int low = 0, mid, high = n - 1;

while (low <= high) {
    mid = (low + high) / 2;

    if (key == arr[mid]) {
        printf("Key : '%d' is present at index %d", key, mid);
        return;
    }
    else if (key < arr[mid]) {
        high = mid - 1;
    }
    else {
        low = mid + 1;
    }
}

printf("'%d' not found", key);
```

```
}

void binarySearchInDescendingArray (int arr[], int n) {

    for (int i = 0; i < n; i++)
    {
        for (int j = i + 1; j < n; j++)
        {
            int temp;
            if (arr[i] < arr[j]) {
                temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
    }

    printf("\n\nDescending Sorted Array : ");
    for (int i = 0; i < n; i++)
    {
        printf("%d ", arr[i]);
    }

    printf("\n");

    int key;
    printf("Enter value of Key : ");
    scanf("%d", &key);
```

```
    int low = 0, mid, high = n - 1;

    while (low <= high) {
        mid = (low + high) / 2;

        if (key == arr[mid]) {
            printf("Key : '%d' is present at index %d", key, mid);
            return;
        }
        else if (key > arr[mid]) {
            high = mid - 1;
        }
        else {
            low = mid + 1;
        }
    }

    printf("'%d' not found", key);

}

void binarySearchInCharacterArray (char arr[], int n) {

    for (int i = 0; i < n; i++)
    {
        for (int j = i + 1; j < n; j++)
        {
            int temp;
```

```
            if (arr[i] > arr[j]) {
                temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }
    }

    printf("\n\n");

    printf("Ascending Sorted in Character Array : ");
    for (int i = 0; i < n; i++)
    {
        printf("%c ", arr[i]);
    }

    printf("\n");

    char key;
    printf("Enter value of Key : ");
    scanf(" %c", &key);

    int low = 0, mid, high = n - 1;

    while (low <= high) {
        mid = (low + high) / 2;

        if (key == arr[mid]) {
            printf("Key : '%c' is present at index %d", key, mid);
```

```
            return;
        }
        else if (key < arr[mid]) {
            high = mid - 1;
        }
        else {
            low = mid + 1;
        }
    }

    printf("'%c' not found", key);

}

int main () {

    int array[] = {29, 10 , 14, 37, 13, 7, 25, 18, 3, 40, 21, 6, 31, 12, 2};
    char arrayCHAR[] = {'a', 'g', 'q', 'w', 'e', 'r', 't', 'y', 'u', 'i', 'o', 'p', 's', 'd',
'f'};
    int size = 15;

    binarySearchInAscendingArray(array, size);
    binarySearchInDescendingArray(array, size);
    binarySearchInCharacterArray(arrayCHAR, size);
}
```

Binary Search Output :-

```
PS D:\ADA-Code\Practical-2> gcc binarysearch.c
PS D:\ADA-Code\Practical-2> ./a.exe

Ascending Sorted Array : 2 3 6 7 10 12 13 14 18 21 25 29 31 37 40
Enter value of Key : 18
Key : '18' is present at index 8

Descending Sorted Array : 40 37 31 29 25 21 18 14 13 12 10 7 6 3 2
Enter value of Key : 18
Key : '18' is present at index 6

Ascending Sorted in Character Array : a d e f g i o p q r s t u w y
Enter value of Key : s
Key : 's' is present at index 10
PS D:\ADA-Code\Practical-2>
```